

Ghi Chú Lý Thuyết Hỗ Trợ Bài Tập

Biến Đổi Hình Thái · Kernel Sobel/Gaussian/Sharpen · Nội Suy

Môn: Xử Lý Ảnh và Thị Giác Máy Tính – 121036

May 14, 2026

1 Phép Biến Đổi Hình Thái

1.1 Ảnh nhị phân và phần tử cấu trúc

Ảnh nhị phân

Một ảnh nhị phân được coi là tập hợp điểm

$$A \subseteq \mathbb{Z}^2,$$

trong đó điểm $(x, y) \in A$ nếu pixel đó là *tiền cảnh* (giá trị 1), còn lại là nền (giá trị 0).

Phần tử cấu trúc (Structuring Element)

Phần tử cấu trúc là một tập hữu hạn $B \subseteq \mathbb{Z}^2$ với một *điểm neo* (anchor) $o \in B$. Trong thực hành, B thường là ma trận nhỏ chứa các ô 0/1.

Ký hiệu dịch chuyển. Khi đặt B tại vị trí $\mathbf{x} = (x_1, x_2)$:

$$B_{\mathbf{x}} = \{ b + \mathbf{x} \mid b \in B \}.$$

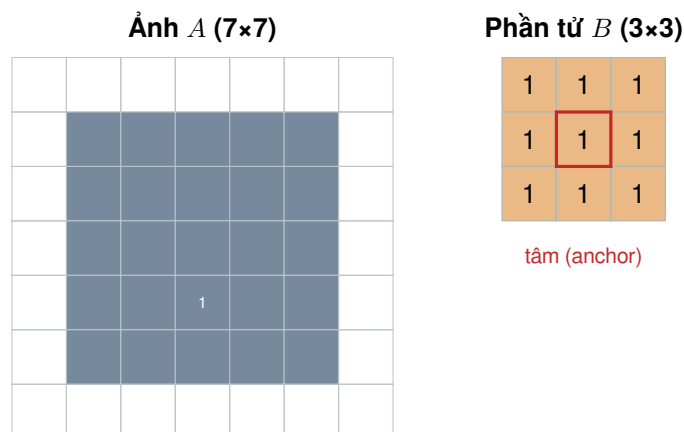


Figure 1: Ảnh nhị phân A (màu xanh = tiền cảnh) và phần tử cấu trúc B hình vuông 3×3 (màu cam). Ô đỏ là điểm neo.

1.2 Phép Co (Erosion)

Phép Co

$$A \ominus B = \{ \mathbf{x} \in \mathbb{Z}^2 \mid B_{\mathbf{x}} \subseteq A \}.$$

Điểm $\mathbf{x}$ thuộc kết quả khi **toàn bộ** B đặt tại $\mathbf{x}$ nằm gọn trong A .

Quy tắc kiểm tra từng pixel. Với ảnh nhị phân, erosion tại pixel (r, c) cho giá trị 1 khi và chỉ khi mọi pixel trong cửa sổ B quanh (r, c) đều bằng 1:

$$(A \ominus B)[r, c] = \min_{(k, l) \in B} A[r + k, c + l].$$

Nói cách khác: lấy *minimum* của cửa sổ lân cận.

Ví dụ số – Erosion 5×5 với B hình vuông 3×3

Ảnh A (1 = trắng, 0 = đen) và kết quả $A \ominus B$:

Ảnh A									
0	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0
0	1	1	1	0	0	0	1	0	0
0	1	1	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$\ominus B_{3 \times 3}$ 

$A \ominus B$									
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Giải thích: Chỉ pixel trung tâm $(2, 2)$ có cửa sổ 3×3 nằm hoàn toàn trong vùng trắng của A . Toàn bộ biên bị “ăn mòn”.

1.3 Phép Giãn (Dilation)

Phép Giãn

$$A \oplus B = \{ \mathbf{x} \in \mathbb{Z}^2 \mid B_{\mathbf{x}} \cap A \neq \emptyset \}.$$

Điểm $\mathbf{x}$ thuộc kết quả khi **ít nhất một** điểm của B đặt tại $\mathbf{x}$ rơi vào tiền cảnh A .

Quy tắc kiểm tra từng pixel.

$$(A \oplus B)[r, c] = \max_{(k, l) \in B} A[r + k, c + l].$$

Nói cách khác: lấy *maximum* của cửa sổ lân cận.

Ví dụ số – Dilation 5×5 với B hình chữ thập 3×3



Điểm (2, 2) “mở rộng” theo hình chữ thập của B – đúng với định nghĩa: mỗi điểm trong B khi đặt tại (2, 2) đều rơi vào A (có ít nhất 1).

1.4 Phép Mở (Opening) và Phép Đóng (Closing)

Phép Mở và Đóng

$$A \circ B = (A \ominus B) \oplus B \quad (\text{erosion, rồi dilation})$$

$$A \bullet B = (A \oplus B) \ominus B \quad (\text{dilation, rồi erosion})$$

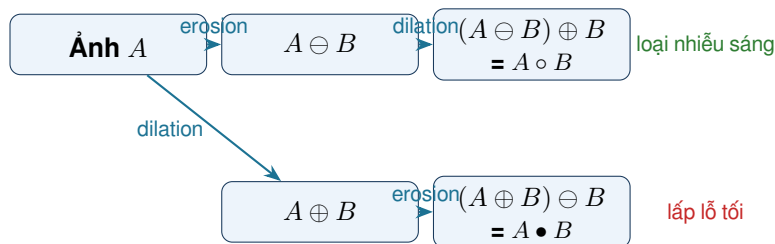


Figure 2: Pipeline của phép mở (hàng trên) và phép đóng (hàng dưới).

Hiệu ứng của phép mở: Loại bỏ vật thể / chi tiết sáng nhỏ hơn B . Vật thể lớn hơn B được bảo toàn (hình dạng gần như không đổi).

Hiệu ứng của phép đóng: Lấp các lỗ tối nhỏ hơn B . Nối các khe hở hẹp trong tiền cảnh.

1.5 Gradient Hình Thái, Top-Hat, Black-Hat

Ba phép kết hợp thường gặp trong bài tập:

Gradient Hình Thái

$$\text{Grad}(A) = (A \oplus B) - (A \ominus B).$$

Kết quả nhấn mạnh *vùng biên* của vật thể.

Top-Hat Transform (ảnh mức xám)

Với ảnh mức xám I :

$$\text{TopHat}(I) = I - (I \circ B).$$

Làm nổi bật chi tiết *sáng nhỏ* hơn B trên nền biến thiên chậm.

Black-Hat Transform

$$\text{BlackHat}(I) = (I \bullet B) - I.$$

Làm nổi bật chi tiết *tối nhỏ* hơn B .

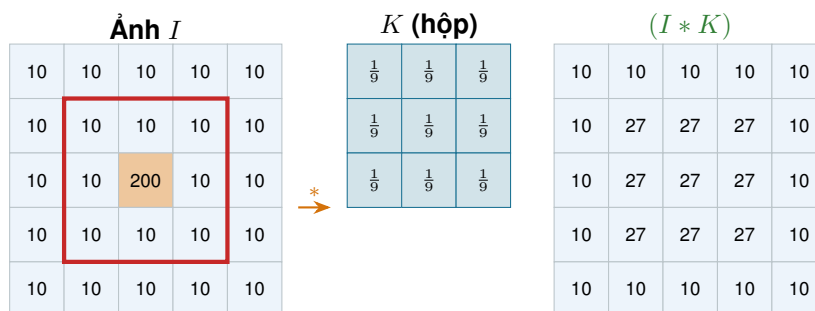
2 Kernel Sobel, Gaussian và Sharpen

2.1 Tích Chập Rời Rạc 2D – Nhắc Lại

Với ảnh xám $I[r, c]$ và kernel $K[m, n]$ kích thước $(2k + 1) \times (2k + 1)$:

$$(I * K)[r, c] = \sum_{m=-k}^k \sum_{n=-k}^k I[r + m, c + n] \cdot K[m, n].$$

Đây là **tổng trọng số** của cửa sổ lân cận – kernel K là bảng trọng số.



$$\frac{200}{9} + \frac{8 \times 10}{9} \approx 27$$

Figure 3: Tích chập: cửa sổ dò trượt qua I , tại mỗi vị trí tính tổng trọng số với K .

2.2 Gaussian Kernel – Từ Công Thức Đến Số

Liên hệ Giải Tích – Giải Tích 2 – Phân phối chuẩn

Bạn đã học phân phối chuẩn trong xác suất:

$$g(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right).$$

Kernel Gaussian là **phiên bản rời rạc hoá** của hàm này.

Bước 1 – Rời rạc hoá Gaussian 1D

Với kernel kích thước $n = 2r + 1$ (số lẻ), lấy mẫu tại $x = -r, \dots, 0, \dots, r$:

$$h[k] = \exp\left(-\frac{k^2}{2\sigma^2}\right), \quad k = -r, \dots, r.$$

Sau đó **chuẩn hoá** (chia cho tổng để tổng = 1):

$$\hat{h}[k] = \frac{h[k]}{\sum_{j=-r}^r h[j]}.$$

Ví dụ số – Gaussian 1D với $\sigma = 1, n = 3$ (tức $r = 1$)

$$h[-1] = e^{-\frac{1}{2}} \approx 0.607, \quad h[0] = e^0 = 1, \quad h[1] = e^{-\frac{1}{2}} \approx 0.607.$$

Tổng $\approx 0.607 + 1 + 0.607 = 2.214$.

Sau chuẩn hoá:

$$\hat{h} \approx [0.274, 0.452, 0.274].$$

Xấp xỉ nguyên đơn giản: nhân với 4 và làm tròn $\Rightarrow [1, 2, 1]$, tổng = 4. Kernel Gaussian 3×1 hay dùng: $\frac{1}{4}[1, 2, 1]$.

Bước 2 – Mở rộng lên 2D nhờ tính tách biệt

Gaussian 2D tách thành tích hai Gaussian 1D:

$$G(m, n; \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{m^2 + n^2}{2\sigma^2}\right) = g(m) \cdot g(n).$$

Do đó kernel 2D có thể viết dưới dạng tích ngoài (outer product):

$$\mathbf{K}_G = \hat{h} \otimes \hat{h}^T = \hat{h}^T \cdot \hat{h},$$

với $\hat{h}$ là vector hàng Gaussian 1D đã chuẩn hoá.

Ví dụ số – Gaussian 3×3 từ $[1, 2, 1]/4$

$$\mathbf{K}_G = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot \frac{1}{4} [1 \quad 2 \quad 1] = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Kiểm tra: tổng tất cả phần tử = $16/16 = 1$. ✓

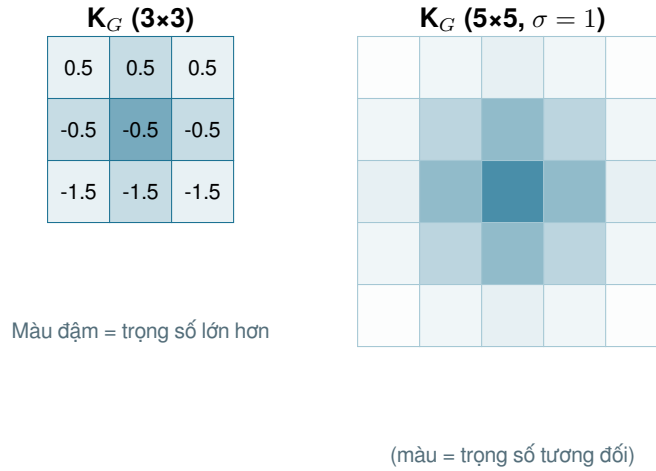


Figure 4: Kernel Gaussian 3×3 (trái) và 5×5 (phải). Trọng số giảm dần từ tâm – đúng với PDF của phân phối chuẩn.

2.3 Sobel Kernel – Từ Đạo Hàm Rời Rạc

Liên hệ Giải Tích – Giải Tích 2 – Đạo hàm và xấp xỉ trung tâm

Từ định nghĩa đạo hàm:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Với pixel, $h = 1$ là bước nhỏ nhất, nên **không thể lấy giới hạn**. Xấp xỉ tốt nhất là *đạo hàm trung tâm* (sai số $O(h^2)$):

$$\left. \frac{\partial I}{\partial c} \right|_{r,c} \approx \frac{I[r, c+1] - I[r, c-1]}{2}.$$

Từ đạo hàm trung tâm đến Sobel

Đạo hàm theo cột (x -direction):

$$\frac{\partial I}{\partial c} \approx \frac{1}{2}(I[r, c+1] - I[r, c-1]) \quad \longleftrightarrow \quad d_c = \frac{1}{2}[-1, 0, +1].$$

Nếu dùng thẳng d_c , kết quả nhạy cảm với nhiễu pixel đơn lẻ. Sobel **làm trơn nhẹ** theo hướng vuông góc (r -direction) trước khi lấy đạo hàm:

$$s_r = [1, 2, 1]^T \quad (\text{Gaussian rời rạc, chưa chuẩn hoá}).$$

Kernel Sobel theo hướng c là **tích ngoài** (outer product):

$$S_c = s_r \otimes d_c = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \cdot [-1 \quad 0 \quad 1] = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}.$$

Tương tự cho hướng r (đạo hàm theo hàng):

$$S_r = d_r \otimes s_c = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \cdot [1 \quad 2 \quad 1] = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

S_c (cạnh đứng)			S_r (cạnh ngang)		
0.5	0.5	0.5	0.5	0.5	0.5
-0.5	-0.5	-0.5	-0.5	-0.5	-0.5
-1.5	-1.5	-1.5	-1.5	-1.5	-1.5

tổng = 0, output = 0 trên vùng đồng đều đỏ = âm, xanh = dương

Figure 5: Kernel Sobel S_c (phát hiện cạnh dọc) và S_r (cạnh ngang). Màu đỏ = trọng số âm, xanh = dương, trắng = 0. **Tổng** toàn bộ phần tử = 0 – vùng hằng số cho output = 0.

Độ mạnh cạnh tổng hợp:

$$|\nabla I|[r, c] = \sqrt{(S_c * I)^2[r, c] + (S_r * I)^2[r, c]}.$$

Ví dụ số – Sobel trên 1 hàng qua cạnh đứng

Hàng pixel: ..., 20, 20, 20, 200, 200, 200, ...

Áp dụng S_c tại vị trí trung tâm của biên (pixel 200 đầu tiên, giả sử $c = 3$):

$$\begin{aligned} (S_c * I)[r, 3] &= (-1) \cdot 20 + 0 \cdot 20 + (+1) \cdot 200 \\ &\quad + (-2) \cdot 20 + 0 \cdot 20 + (+2) \cdot 200 \\ &\quad + (-1) \cdot 20 + 0 \cdot 20 + (+1) \cdot 200 = 720. \end{aligned}$$

Tại vùng đồng đều (tất cả = 20):

$$(S_c * I)[r, 1] = (-1 - 2 - 1) \cdot 20 + (1 + 2 + 1) \cdot 20 = -80 + 80 = 0. \checkmark$$

2.4 Sharpen Kernel – Từ Khai Triển Taylor

Liên hệ Giải Tích – Giải Tích 2 – Khai triển Taylor bậc 2

Khai triển Taylor bậc 2 tại x (với bước $h = 1$):

$$f(x+1) = f(x) + f'(x) + \frac{1}{2}f''(x) + O(1),$$

$$f(x-1) = f(x) - f'(x) + \frac{1}{2}f''(x) + O(1).$$

Cộng hai phương trình, số hạng bậc lẻ triệt tiêu:

$$f(x+1) + f(x-1) = 2f(x) + f''(x) + O(1).$$

Giải ra $f''(x)$:

$$f''(x) \approx f(x-1) - 2f(x) + f(x+1).$$

Laplacian rời rạc 2D

Mở rộng lên 2D – cộng đạo hàm bậc hai theo cả hai chiều:

$$\begin{aligned}\nabla^2 I[r, c] &= \underbrace{I[r, c-1] + I[r, c+1] - 2I[r, c]}_{\partial^2 / \partial c^2} + \underbrace{I[r-1, c] + I[r+1, c] - 2I[r, c]}_{\partial^2 / \partial r^2} \\ &= I[r-1, c] + I[r+1, c] + I[r, c-1] + I[r, c+1] - 4I[r, c].\end{aligned}$$

Kernel Laplacian tương đương:

$$\mathbf{K}_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Từ Laplacian đến Sharpen

Ý tưởng: Laplacian lớn tại cạnh $\Rightarrow$ cộng Laplacian vào ảnh để khuếch đại cạnh:

$$I_{\text{sắc}} = I - \lambda \cdot \nabla^2 I, \quad \lambda > 0.$$

Với $\lambda = 1$, ghép thành một kernel duy nhất:

$$\mathbf{K}_{\text{sắc}} = \mathbf{I}_{\text{kern}} - \mathbf{K}_{\nabla^2} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

$$\begin{array}{c} \mathbf{K}_{\nabla^2} \\ \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ -0.5 & -0.5 & -0.5 \\ -1.5 & -1.5 & -1.5 \end{bmatrix} \end{array} - \begin{array}{c} \mathbf{K}_{\text{sắc}} = \mathbf{I}_{\text{kern}} - \mathbf{K}_{\nabla^2} \\ \begin{bmatrix} 0.5 & 0.5 & 0.5 \\ -0.5 & -0.5 & -0.5 \\ -1.5 & -1.5 & -1.5 \end{bmatrix} \end{array}$$

Tổng = 1 $\rightarrow$ giữ độ sáng trung bình
Tâm = 5, láng giềng = -1

Figure 6: Xây dựng kernel sắc nét từ Laplacian. Tâm nhân (5) làm pixel nổi bật hơn, 4 láng giềng (-1) triệt tiêu phần phẳng.

Tổng quát hoá kích thước. Để xây dựng kernel làm sắc kích thước $n \times n$ bất kỳ:

1. Xây Gaussian $n \times n$ có tổng = 1 (gọi là $\mathbf{H}$).
2. Tính $\mathbf{K}_{\text{sắc}} = \mathbf{I}_{\text{kern}} - \mathbf{H}$, trong đó $\mathbf{I}_{\text{kern}}$ là kernel delta (1 ở tâm, còn lại 0).

Điều này cho phần tử tâm dương, xung quanh âm, *tổng* = 0 (nếu không cộng thêm $\mathbf{I}_{\text{kern}}$) hoặc = 1 (nếu cộng).

3 Biến Đổi Hình Học, Nội Suy và Chống Răng Cưa

Sơ đồ tổng quan của bài tập 2. Tất cả kỹ thuật trong phần này đều xoay quanh một câu hỏi: sau khi ánh xạ ngược đưa pixel đích về tọa độ thực (x, y) trong ảnh nguồn, làm thế nào để ước lượng giá trị tại điểm đó? Câu trả lời khác nhau ở *kernel* được dùng – và quan trọng hơn, ở chỗ kernel đó **cố định** hay **thay đổi theo từng pixel**.

Kỹ thuật	Cơ chế	Kernel	Dùng gì?
Nearest-neighbor	làm tròn tọa độ	$[1]$ delta	—
Bilinear	tích có trọng số 2×2	hat function	tích vô hướng cục bộ
SSAA L	trung bình L^2 mẫu bilinear	box $L \times L$	<code>convolve2d_manual</code>
Bicubic	cubic 4×4 , thay đổi theo δ	$R(\delta - i)$	tích vô hướng cục bộ
Lanczos a	sinc cửa sổ, thay đổi theo δ	$L(\delta - i)$	tích vô hướng cục bộ

3.1 Warping Ngược và Xử Lý Ngoài Biên

Warping ngược (Inverse Warp)

Với mỗi pixel đích (r', c') , tính tọa độ nguồn:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = M^{-1} \cdot \begin{bmatrix} c' \\ r' \\ 1 \end{bmatrix}.$$

Nếu (x, y) nằm ngoài biên ảnh nguồn: gán `fill_value`. Nếu trong biên: ước lượng $I(y, x)$ bằng một trong các phương pháp dưới đây.

Tại sao warping ngược, không phải xuôi? Warping xuôi ánh xạ mỗi pixel nguồn sang đích – có thể nhiều pixel cùng đổ vào một điểm đích, hoặc một điểm đích không có pixel nào đến. Warping ngược đảm bảo mọi pixel đích đều được tính đúng một lần.



Figure 7: Warping ngược: pixel đích (r', c') ánh xạ về tọa độ thực (y, x) . Ô xám là vùng `fill_value` – nguyên nhân viền đen khi xoay ảnh.

3.2 Tích Vô Hướng Cục Bộ vs. Tích Chập Toàn Ảnh

Đây là phân biệt trung tâm của bài tập.

Tích chập toàn ảnh – kernel cố định

$$(I * K)[r, c] = \sum_{m=-k}^k \sum_{n=-k}^k I[r+m, c+n] \cdot K[m, n], \quad K \text{ không đổi theo } (r, c).$$

Dùng `convolve2d_manual`: trượt một kernel *duy nhất* qua toàn ảnh.
Ví dụ: làm mờ Gaussian, Sobel, box filter của SSAA.

Tích vô hướng cục bộ – kernel thay đổi theo δ

$$P(r, c) = \sum_i \sum_j I[r_0 + i, c_0 + j] \cdot K_{r,c}[i, j], \quad K_{r,c} \text{ khác nhau tại mỗi } (r, c).$$

Không thể gọi `convolve2d_manual` một lần duy nhất. Kernel được xây tại chỗ bên trong vòng lặp pixel.
Ví dụ: bicubic, Lanczos.

Tại sao kernel thay đổi? Tọa độ nguồn $(y, x) = M^{-1}[c', r', 1]^T$ thường không nguyên. Phần lẻ $\delta_r = y - \lfloor y \rfloor$ và $\delta_c = x - \lfloor x \rfloor$ thay đổi theo từng pixel đích – và cả bicubic lẫn Lanczos đều dùng (δ_r, δ_c) để xây trọng số. Đây chính là điểm khiến chúng là *lọc phi dừng* (non-stationary), không phải tích chập.

Tích vô hướng cục bộ hoạt động thế nào. Đặt $r_0 = \lfloor y \rfloor$, $c_0 = \lfloor x \rfloor$. Với mỗi cặp offset (i, j) trong cửa sổ, nhân $I[r_0 + i, c_0 + j]$ với trọng số $w_r[i] \cdot w_c[j]$, rồi cộng lại:

$$P = \sum_i \sum_j I[r_0 + i, c_0 + j] \cdot w_r[i] \cdot w_c[j] = \mathbf{w}_r^T \cdot \mathbf{P}_{\text{patch}} \cdot \mathbf{w}_c,$$

trong đó $\mathbf{P}_{\text{patch}}$ là ma trận pixel của cửa sổ. Đây là phép nhân ma trận – không cần lật kernel.

3.3 Nearest-Neighbor và Bilinear – Tích Vô Hướng Đơn Giản Nhất

Nearest-Neighbor

$$I_{\text{NN}}(y, x) = I(\lfloor y + 0.5 \rfloor, \lfloor x + 0.5 \rfloor).$$

Trọng số: $w = [1]$ – tích vô hướng với cửa sổ 1×1 . Không cần tính δ .

Bilinear – cửa sổ 2×2

Đặt $\delta_r = y - \lfloor y \rfloor$, $\delta_c = x - \lfloor x \rfloor$, $Q_{ij} = I[\lfloor y \rfloor + i, \lfloor x \rfloor + j]$:

$$I_{\text{BL}} = (1 - \delta_r)(1 - \delta_c) Q_{00} + (1 - \delta_r)\delta_c Q_{01} + \delta_r(1 - \delta_c) Q_{10} + \delta_r\delta_c Q_{11}.$$

Trọng số hàng: $\mathbf{w}_r = [1 - \delta_r, \delta_r]$; trọng số cột: $\mathbf{w}_c = [1 - \delta_c, \delta_c]$. Tích ngoài: $K = \mathbf{w}_r^T \mathbf{w}_c$ (ma trận 2×2). Tổng trọng số = 1 luôn đúng.

Ví dụ số – Bilinear tại $(y, x) = (1.3, 2.7)$

	$c = 2$	$c = 3$
$r = 1$	80	120
$r = 2$	40	60

$\delta_r = 0.3, \delta_c = 0.7$.

Trọng số: $\mathbf{w}_r = [0.7, 0.3], \mathbf{w}_c = [0.3, 0.7]$.

$R_0 = 0.3 \cdot 80 + 0.7 \cdot 120 = 108, \quad R_1 = 0.3 \cdot 40 + 0.7 \cdot 60 = 54$.

$I_{BL} = 0.7 \cdot 108 + 0.3 \cdot 54 = \mathbf{91.8}$.

Nearest-neighbor cho $I[1, 3] = 120$ – khác 28 đơn vị.

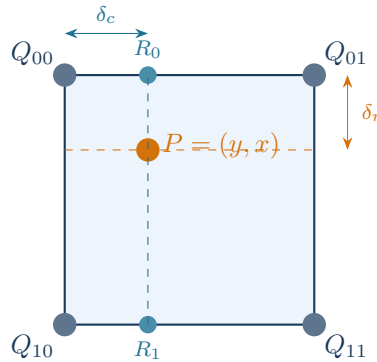


Figure 8: Bilinear: hai lần tích vô hướng 1D. Bước 1: nội suy R_0, R_1 theo δ_c . Bước 2: nội suy P từ R_0, R_1 theo δ_r .

3.4 SSAA – Kernel Cố Định, Dùng `convolve2d_manual`

SSAA là kỹ thuật duy nhất trong bài tập có thể cài đặt bằng `convolve2d_manual` trực tiếp – vì kernel của nó *không phụ thuộc* vào (δ_r, δ_c) .

Ý tưởng

Lấy L^2 mẫu đều nhau bên trong diện tích pixel đích, ánh xạ ngược từng mẫu về ảnh nguồn, nội suy bilinear, rồi trung bình:

$$C(r', c') = \frac{1}{L^2} \sum_{m=0}^{L-1} \sum_{n=0}^{L-1} f\left(r' + \frac{m+0.5}{L}, c' + \frac{n+0.5}{L}\right).$$

Liên hệ với bộ lọc hộp và `convolve2d_manual`

SSAA tương đương áp dụng bộ lọc hộp $L \times L$ trên ảnh đã warp:

$$K_{\text{box}}^{(L)} = \frac{1}{L^2} \mathbf{1}_{L \times L}.$$

Kernel này **cố định** – không phụ thuộc vào vị trí pixel. Do đó có thể gọi `convolve2d_manual` một lần:

1. Warp ảnh bằng `warp_affine_manual` với `nearest-neighbor`.
2. Gọi `convolve2d_manual(warped, K_box)` để làm mượt.

Ví dụ số – Kernel hộp 2×2 và 3×3

$$K_{\text{box}}^{(2)} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} \end{bmatrix}$$

tổng = 1, $L = 2$

$$K_{\text{box}}^{(3)} = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

tổng = 1, $L = 3$

`np.ones((L,L))/L**2`
`out = convolve2d_manual(warped, K)`

Đây là tích chập thực sự:
 kernel không đổi theo pixel.

3.5 Bicubic – Tích Vô Hướng Cục Bộ 4×4

Hàm trọng số bicubic 1D – $R(d)$

$$R(d) = \begin{cases} (a+2)|d|^3 - (a+3)|d|^2 + 1 & |d| \leq 1 \\ a|d|^3 - 5a|d|^2 + 8a|d| - 4a & 1 < |d| < 2 \\ 0 & |d| \geq 2, \end{cases} \quad a = -0.5 \text{ (Keys)}.$$

$R(0) = 1, R(1) = 0$, vùng âm nhẹ $1 < |d| < 2$ tăng độ sắc nét.

Xây kernel tại điểm (y, x)

1. Tính $\delta_r = y - \lfloor y \rfloor, \delta_c = x - \lfloor x \rfloor$.
2. Lấy $i \in \{-1, 0, 1, 2\}$: $w_r[i] = R(\delta_r - i)$.
3. Lấy $j \in \{-1, 0, 1, 2\}$: $w_c[j] = R(\delta_c - j)$.
4. Kernel 4×4 : $K[i, j] = w_r[i] \cdot w_c[j]$ (tích ngoài).

Tích vô hướng cục bộ

$$P(y, x) = \sum_{i=-1}^2 \sum_{j=-1}^2 I[\lfloor y \rfloor + i, \lfloor x \rfloor + j] \cdot R(\delta_r - i) \cdot R(\delta_c - j) = \mathbf{w}_r^T \cdot \mathbf{P}_{4 \times 4} \cdot \mathbf{w}_c.$$

Tại sao không thể dùng `convolve2d_manual`? w_r và w_c phụ thuộc vào δ_r, δ_c – khác nhau tại mỗi pixel đích. Không có một kernel cố định nào áp dụng cho toàn ảnh. Cần tính K mới bên trong vòng lặp mỗi pixel.

Ví dụ số – Bảng $R(d)$ tại các điểm đặc biệt, $a = -0.5$

d	Công thức dùng	$R(d)$
0	$(1.5)(0) - (2.5)(0) + 1$	1
0.5	$(1.5)(0.125) - (2.5)(0.25) + 1$	0.5625
1	$(1.5)(1) - (2.5)(1) + 1$	0
1.5	$(-0.5)(3.375) - 5(-0.5)(2.25) + 8(-0.5)(1.5) - 4(-0.5)$	-0.0625
2	ngoài phạm vi	0

Giá trị âm tại $d = 1.5$ là “lobe âm” – tăng độ sắc nét nhẹ.

Tính tách biệt (separability). Vì $K[i, j] = w_r[i] \cdot w_c[j]$, có thể tính trong 2 bước 1D: $T_i = \sum_j I[\cdot + i, \cdot + j] \cdot w_c[j]$, rồi $P = \sum_i T_i \cdot w_r[i]$. Chi phí giảm từ $4^2 = 16$ xuống $4 + 4 = 8$ phép nhân.

3.6 Lanczos – Tích Vô Hướng Cục Bộ $(2a) \times (2a)$

Liên hệ Giải Tích – Hàm sinc

$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$, $\text{sinc}(0) = 1$. Đây là bộ lọc thông thấp lý tưởng trong lý thuyết tín hiệu, nhưng có vùng hỗ trợ vô hạn – cần “cửa sổ hoá” để dùng trong thực tế.

Hàm trọng số Lanczos 1D – $L(x)$

$$L(x) = \begin{cases} 1 & x = 0 \\ \text{sinc}(x) \cdot \text{sinc}(x/a) & 0 < |x| < a \\ 0 & |x| \geq a. \end{cases}$$

Tham số $a = 2$ hoặc $a = 3$. Với $a = 3$: cửa sổ 6×6 pixel.

Xây kernel tại điểm (y, x) và chuẩn hoá

1. Tính $\delta_r = y - \lfloor y \rfloor$, $\delta_c = x - \lfloor x \rfloor$.
2. Với $i \in \{-a+1, \dots, a\}$: $w_r[i] = L(\delta_r - i)$.
3. Với $j \in \{-a+1, \dots, a\}$: $w_c[j] = L(\delta_c - j)$.
4. Tích vô hướng cục bộ và chuẩn hoá:

$$P(y, x) = \frac{\sum_i \sum_j I[\lfloor y \rfloor + i, \lfloor x \rfloor + j] \cdot w_r[i] \cdot w_c[j]}{\sum_i \sum_j w_r[i] \cdot w_c[j]}.$$

Tại sao cần chuẩn hoá? Gần biên ảnh, một số pixel $I[\cdot + i, \cdot + j]$ nằm ngoài – phải bỏ qua. Tổng trọng số còn lại nhỏ hơn 1, nếu không chuẩn hoá ảnh sẽ tối dần về phía biên.

Tại sao không thể dùng `convolve2d_manual`? Cùng lý do như bicubic: w_r, w_c phụ thuộc δ_r, δ_c của từng pixel đích. Kernel không cố định, không thể trượt toàn ảnh.

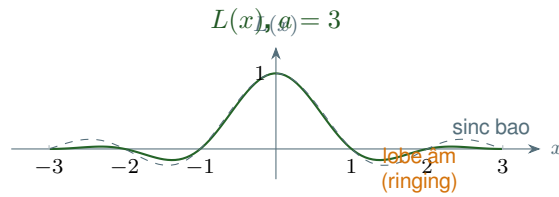


Figure 9: Kernel Lanczos $a = 3$. Lobe âm gần $x = \pm 1, \pm 2$ gây halo mờ gần cạnh tương phản cao.

3.7 Tại Sao Có Răng Cưa Và Mỗi Kỹ Thuật Xử Lý Ra Sao

Liên hệ Giải Tích – Giải Tích 2 – Định lý Nyquist

Hàm liên tục $f(x)$ được lấy mẫu tại lưới nguyên $f(0), f(1), \dots$. Nếu f dao động nhanh hơn $\frac{1}{2}$ chu kỳ/pixel (tần số Nyquist), không thể tái tạo chính xác – xuất hiện aliasing. Cạnh sắc và đường chéo trong ảnh sau warp chứa tần số cao vượt Nyquist.

Mỗi kỹ thuật “làm mượt” theo cách khác nhau:

- **Nearest**: không làm mượt gì – răng cưa nguyên vẹn.
- **Bilinear**: hạt kernel 2×2 – giảm aliasing nhẹ, có thể làm mờ cạnh.
- **SSAA**: box filter – trung bình nhiều mẫu, giảm aliasing tốt nhưng tốn tài nguyên L^2 lần.
- **Bicubic**: cubic kernel 4×4 có lobe âm – ít mờ hơn bilinear, giữ cạnh tốt hơn.
- **Lanczos**: sinc cửa sổ $(2a) \times (2a)$ – ít aliasing nhất, nhưng có thể tạo halo (ringing) mờ.

3.8 Bảng So Sánh Tổng Hợp

Tiêu chí	Nearest	Bilinear	SSAA	Bicubic	Lanczos
Cửa sổ	1×1	2×2	—	4×4	$(2a) \times (2a)$
Phép nhân/pixel	1	4	$L^2 \times 4$	16	$(2a)^2$
Kernel cố định?	có	có	có	không	không
Dùng <code>convolve2d</code> ?	—	—	✓	không	không
Cơ chế	làm tròn	tích vô hướng	tích chập	tích vô hướng	tích vô hướng
Răng cưa	nhiều	vừa	ít	ít	ít nhất
Làm mờ biên	không	vừa	vừa	ít	rất ít
Ringings (halo)	không	không	không	không	có thể

4 Tóm Tắt Công Thức Quan Trọng

Phép toán	Công thức	Quy tắc nhanh
Erosion (pixel)	$\min_{(k,l) \in B} I[r+k, c+l]$	minimum cửa sổ
Dilation (pixel)	$\max_{(k,l) \in B} I[r+k, c+l]$	maximum cửa sổ
Opening	$(A \ominus B) \oplus B$	loại chi tiết sáng nhỏ
Closing	$(A \oplus B) \ominus B$	lấp lỗ tối nhỏ
Tích chập	$\sum_{m,n} I[r+m, c+n] \cdot K[m, n]$	tổng trọng số
Gaussian 1D	$h[k] \propto e^{-k^2/(2\sigma^2)}$ rồi chuẩn hoá	tổng $K = 1$
Sobel S_c	$[1, 2, 1]^T \otimes [-1, 0, 1]$	tổng $K = 0$
Laplacian	$f[x-1] - 2f[x] + f[x+1]$ (mỗi chiều)	từ Taylor bậc 2
Sharpen	$I_{\text{kern}} - K_{\nabla^2}$	tổng $K = 1$
NN	$I[\lfloor y + 0.5 \rfloor, \lfloor x + 0.5 \rfloor]$	làm tròn
Bilinear	$\mathbf{w}_r^T \mathbf{P}_{2 \times 2} \mathbf{w}_c, w = [1-\delta, \delta]$	tích vô hướng 2×2
SSAA L	<code>convolve2d_manual($I_{\text{warp}}, K_{\text{box}}^{(L)}$)</code>	tích chập, kernel cố định
Bicubic	$\mathbf{w}_r^T \mathbf{P}_{4 \times 4} \mathbf{w}_c, w_i = R(\delta-i)$	tích vô hướng 4×4 , biến đổi
Lanczos	$\sum_{i,j} I \cdot L(\delta_r-i) \cdot L(\delta_c-j) / \text{tổng}$	tích vô hướng $(2a)^2$, chuẩn hoá
Phép toán	Công thức	Quy tắc nhanh
Box filter $L \times L$	$K_{ij} = 1/L^2$	tổng $K = 1$, dùng <code>convolve2d</code>
SSAA hệ số L	<code>convolve2d_manual($I_{\text{warp}}, K_{\text{box}}^{(L)}$)</code>	tích chập toàn ảnh
Bicubic weight	$R(d)$: Keys cubic, $R(0) = 1, R(1) = 0$	tích vô hướng cục bộ
Lanczos weight	$L(x) = \text{sinc}(x) \cdot \text{sinc}(x/a)$	tích vô hướng cục bộ
Chuẩn hoá Lanczos	Chia cho $\sum \sum L(\delta_r-i) \cdot L(\delta_c-j)$	bắt buộc gần biên